THE UNIVERSITY OF MELBOURNE

COMP30023 – Computer Systems

Processes and Threads: Introduction

Maria Rodriguez Read

Semester 1, 2025

© University of Melbourne

Process

- A running program
- Fundamental abstraction provided by the OS – provides programs with their own dedicated, abstract machine to run on
- Program
  - code, static.
  - E.g., a recipe
- Process
  - execution of the program, dynamic.
  - E.g., cooking
- Multiple processes can run the same program

Elements of a Process

- Threads -> abstract the CPU
- Address space -> abstract memory
- Files -> abstract disks
- Sockets -> abstract networks

© University of Melbourne

Why Processes?

- Support the ability to run multiple programs *simultaneously* on a single *CPU*

Multiprogramming

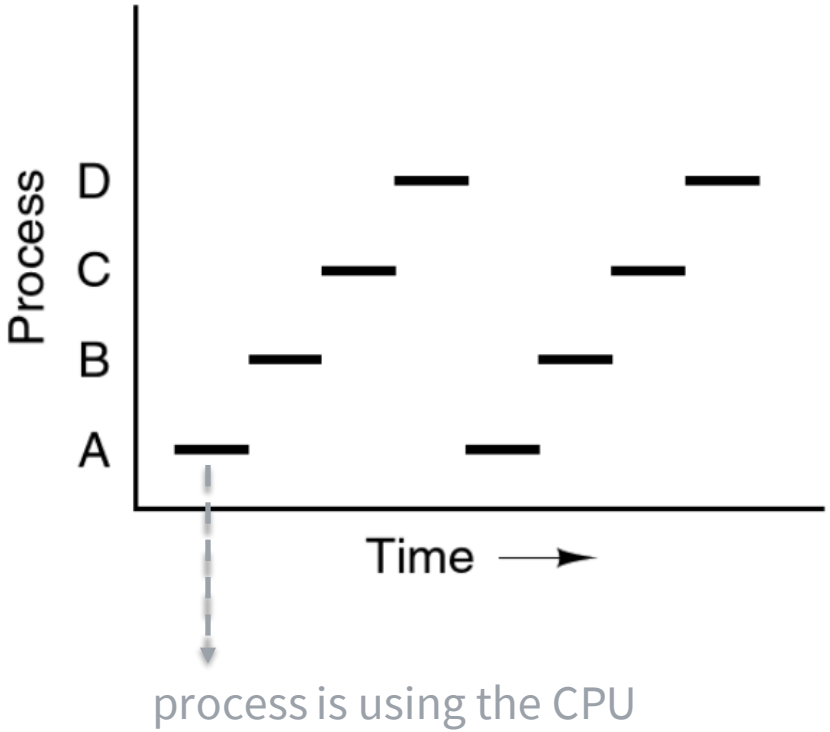
- Allows multiple programs to share a CPU and run in (pseudo) parallel
- CPUs can only run one program at a time – OS provides the illusion of parallelism
- How?
  - The CPU switches from process to process quickly (e.g., each process runs for tens of milliseconds)

© University of Melbourne

Multiprogramming

- Process execution is interleaved
- Provides the illusion of parallelism
- Allows for increased efficiency
- Scheduling algorithm determines which process gets to run next and for how long

Based on TB Figure 2-1



Multiprogramming

- Demonstration of multiprogramming

© University of Melbourne

Process Creation

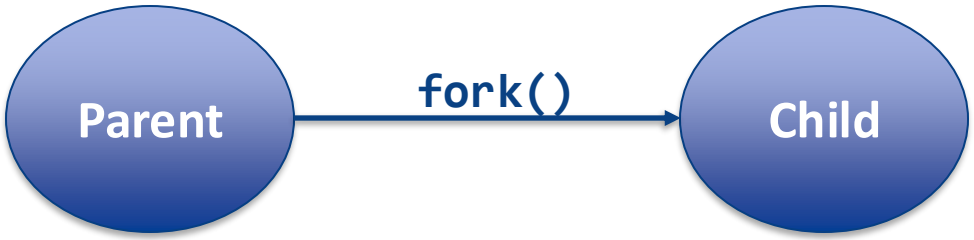
Four main events that cause processes to be created:

1. System initialisation
2. Execution of a process creation system call by a running process
3. A user request to create a new process
4. Initiation of a batch job

© University of Melbourne

Process Creation: Example

- System call in UNIX: **`fork()`**
- Creates a clone of the calling process
  - Different address spaces
  - Same memory image
  - Same values for the program counter and general-purpose registers, same open file handles



© University of Melbourne

Process Creation: Example

- What happens after **`fork()`**?

```c
int main()
{
    fork();
    printf("Hello world!\n");
    return 0;
}
```

```c
int main()
{
    if (fork() == 0)
        printf("Child\n");
    else
        printf("Parent\n");
    return 0;
}
```

- **`fork()`** is usually followed by an invocation to a function in the **`exec()`** family, e.g.,
  - **`execve()`** causes the program that is currently being run by the calling process to be replaced with a new program, with newly initialized stack, heap, and data segments (from the linux man pages)

Process Termination

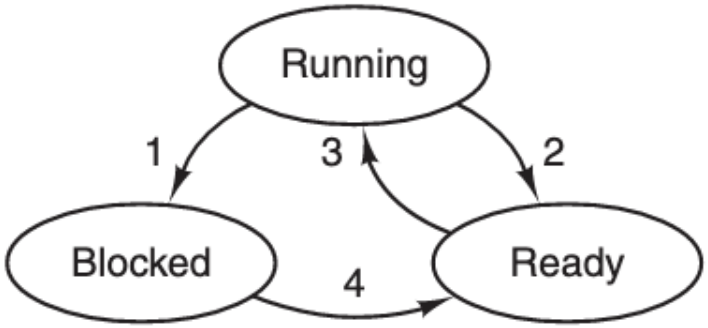
Typical conditions that terminate a process:

1. Normal exit (voluntary)
2. Error exit (voluntary)
3. Fatal error (involuntary)
4. Killed by another process (involuntary)

© University of Melbourne

Process States

- A process may be in one of three states:
  - Running: using the CPU
  - Ready: runnable, but temporarily stopped
  - Blocked: unable to run until some external event happens



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

TB Figure 2-2

Example of State Transition

Time	$Process_0$	$Process_1$	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	$Process_0$ initiates I/O
4	Blocked	Running	$Process_0$ is blocked,
5	Blocked	Running	so $Process_1$ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	$Process_1$ now done
9	Running	–	
10	Running	–	$Process_0$ now done

Figure 4.4 in Operating Systems: Three Easy Pieces, Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau , Arpaci-Dusseau Books, 2023.

© University of Melbourne

Process Control Block

- The OS maintains information about a process in a data structure called the Process Control Block (PCB)
- Process table: list of PCBs (one PCB for each process)
- Examples of data stored in the PCB include:
  - Process ID, process state, parent process, memory management information, file descriptors, priority, used CPU time, etc.
  - What about the execution context? E.g., program counter, stack pointer, other control registers?

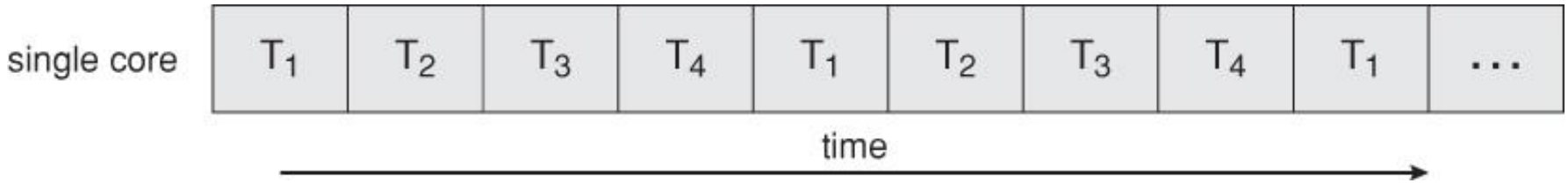
© University of Melbourne

Threads

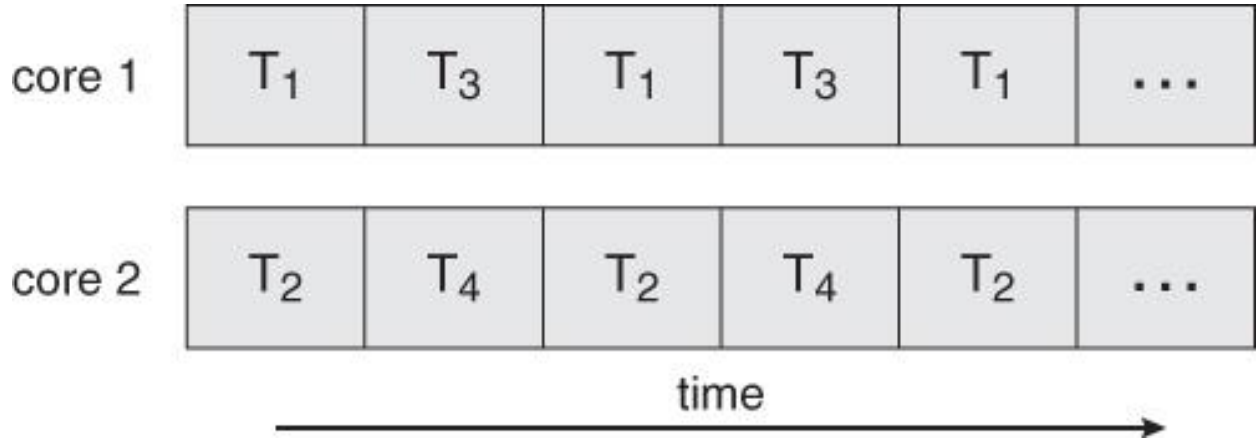
- A thread provides an execution context to a process
  - Sequential execution of a set of instructions
  - Has a program counter, stack pointer, registers, stack
- To run, the execution context of a thread is loaded into the CPU's registers:
  - Program counter, stack pointer, general purpose registers with intermediate values for ongoing computations
- When not running, the context of a thread does not reside in the CPU. Instead, the thread's context is stored in memory, and the processor's registers hold the context for some other thread. E.g.,
  - PC register points to an instruction of some other thread
  - SP register points to the stack of some other thread

© University of Melbourne

Multi-threaded Execution



Silberschatz et al. Figure 4-3



Silberschatz et al. Figure 4-4

Threads

- Execute within the environment defined by a process
- A process has one or more threads

Per-process items	Per-thread items
Address space	Program counter
Global variables	Registers
Open files	Stack
Child processes	State
Pending alarms	
Signals and signal handlers	
Accounting information	

TB Figure 2-12

Threads

- Demonstration of multi-threaded program

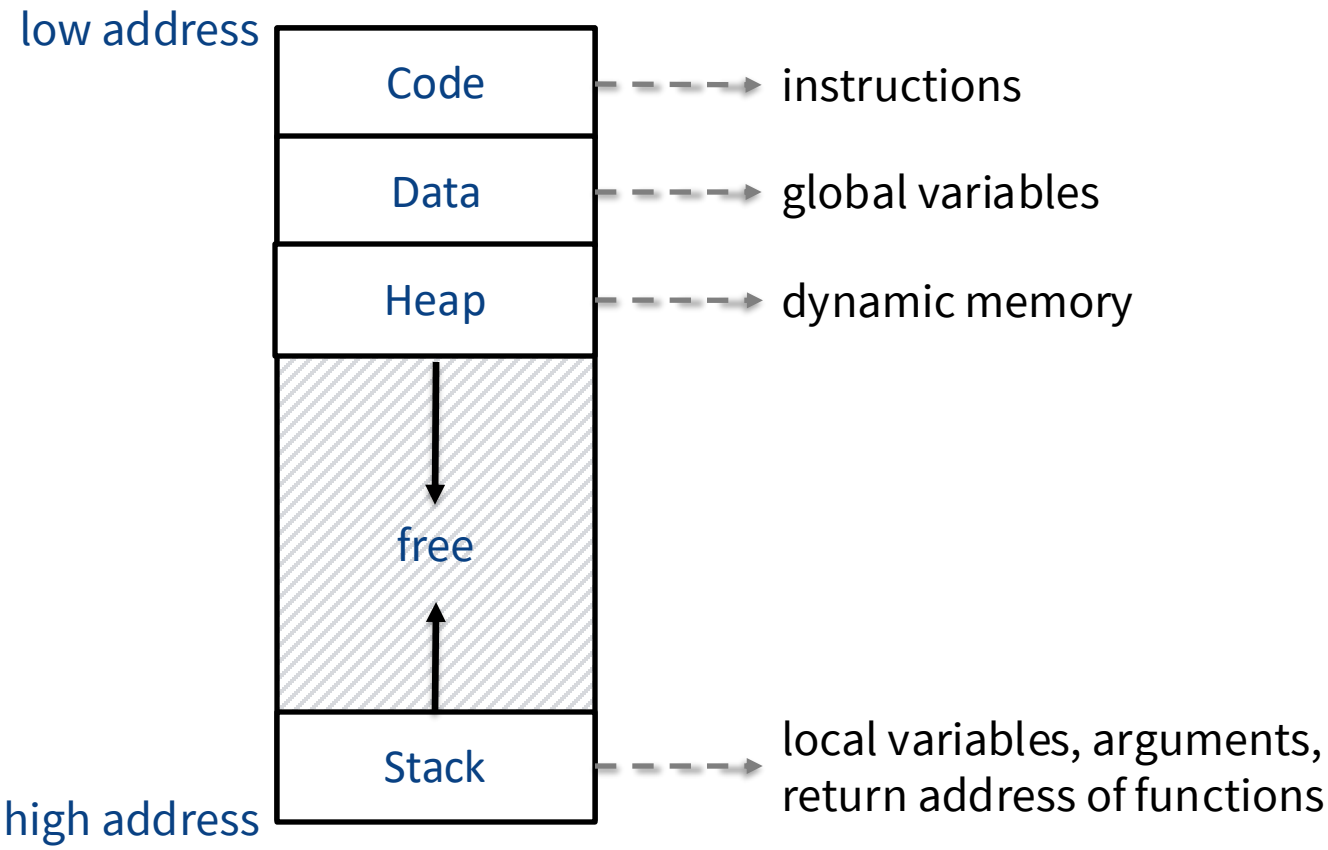
© University of Melbourne

Address Space of Threads

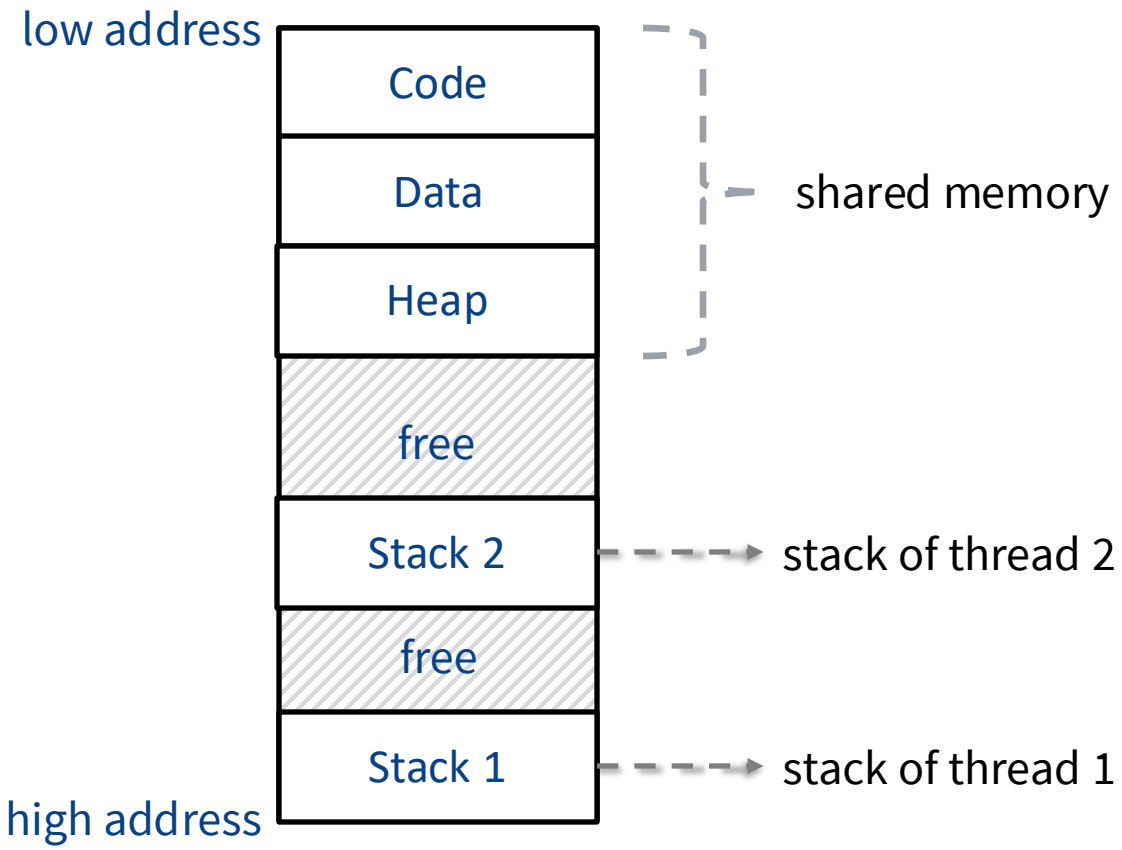
- All threads of a process share the same address space
- Code, data and heap memory are shared between threads
- Each thread has its own stack

© University of Melbourne

Memory in a Single-Threaded Process



Memory in a Multi-Threaded Process



Why Threads?

- Parallelism
  - Through multi-threading, a program has the potential to use multiple processors to each perform a portion of the work
  - Can speed up the completion of a program
  - E.g., performing operations on a very large array -> each thread operates on independent sections of the array
- Enable overlap of I/O with other activities
  - Avoid blocking progress while waiting for I/O operations
  - One thread waits for I/O, while another thread does something useful with the CPU
  - E.g., web servers, databases

© University of Melbourne

Multiple Threads or Multiple Processes?

- Threads
  - Share an address space.
    - Makes sharing data easy and fast,
    - Does not provide isolation, e.g., one thread can bring the entire process down
  - Are created faster
  - Incur in less overhead when switching between threads
- Processes
  - Enable applications to be parallelised beyond the capacity of a single machine
  - Provide isolated execution environments
  - Incur in interprocess communication overhead

© University of Melbourne

Thread Control Block

- Processes: context data stored in PCBs
- Threads: context data stored in Thread Control Blocks (TCBs)
- Examples of stored data include:
  - Thread ID
  - Stack pointer
  - Program counter
  - Register values
  - State (e.g., running, blocked, ready)
  - Pointer to PCB of the process to which the thread belongs to

© University of Melbourne

PThreads

A POSIX standard API for thread creation and synchronisation (most UNIX systems support it)

- functions start with `pthread`
- include `pthread.h`
- threads have an id of type `pthread_t`

Thread call	Description
Pthread_create	Create a new thread
Pthread_exit	Terminate the calling thread
Pthread_join	Wait for a specific thread to exit
Pthread_yield	Release the CPU to let another thread run
Pthread_attr_init	Create and initialize a thread's attribute structure
Pthread_attr_destroy	Remove a thread's attribute structure

TB Figure 2-14